

Arquitectures Big Data

Francesc Barragán Pascual

Darrera revisió: 27.01.2026



 Generalitat de Catalunya Departament d'Educació i FP	CE IA&BD	
M5075 BigData Aplicat		Arquitectures de BigData

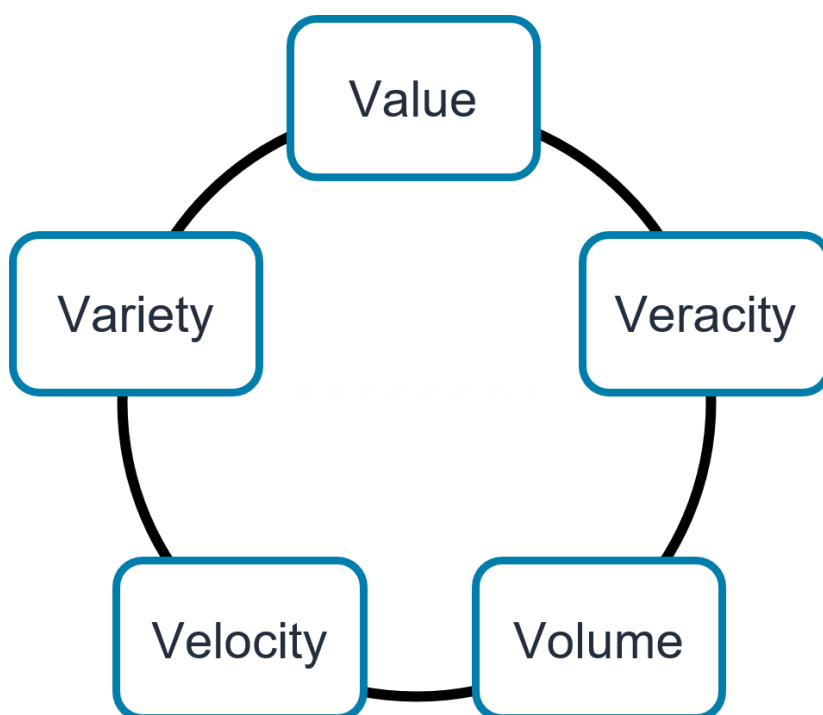
Índex

Índex	2
Introducció	3
Característiques	3
WAF	4
Tipus d'arquitectures	5
Processament Batch	5
Processament en Streaming	6
Arquitectura Lambda	6
Arquitectura Kappa	9
Arquitectura Delta	11
Arquitectura per capes	12
Tecnologies	13
Principi SCV	13
Casos d'ús	15
Bones pràctiques	16
Referències	17

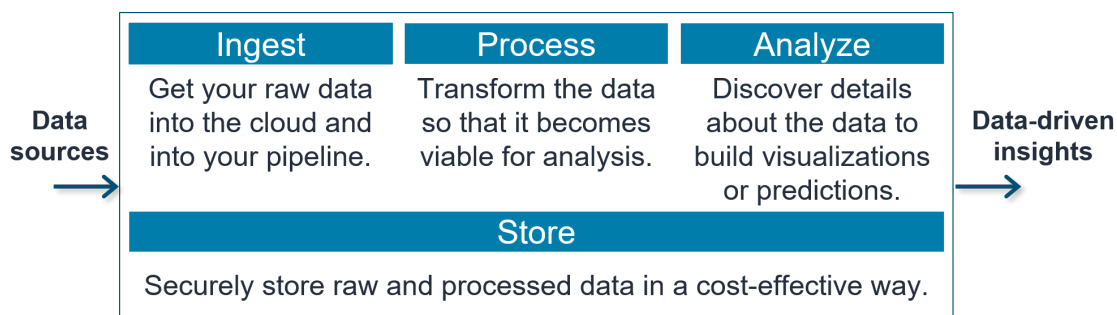
 Generalitat de Catalunya Departament d'Educació i FP	CE IA&BD	
M5075 BigData Aplicat		Arquitectures de BigData

Introducció

Ja sabem en què consisteix Big Data i que, dins dels seus 5V, dues de les més importants són el **volum** i la **velocitat** (les altres: varietat, veracitat i valor). Per complir amb aquestes necessitats, necessitem una infraestructura que doti les nostres aplicacions de tota la potència i robustesa necessàries.



Una arquitectura de *Big Data* es dissenya per gestionar **la ingesta, el processament i l'anàlisi de les dades** que són massa grans o complexes per a un sistema tradicional de base de dades.



 Generalitat de Catalunya Departament d'Educació i FP	CE IA&BD	
M5075 BigData Aplicat		Arquitectures de BigData

Entenem el concepte d'arquitectura de dades com el disseny dels sistemes que donen suport a les necessitats de les dades en una empresa, facilitant la seva evolució i manteniment.

Fins ara no hem aprofundit en cap tecnologia concreta, ja que l'*stack* d'eines és molt ampli i en constant creixement. Al llarg del curs anirem coneixent les diferents eines i aprendrem com i quan utilitzar-les.

Característiques

Totes les arquitectures que dissenyem / utilitzem han de complir les següents característiques:

- **Escalabilitat**: permet augmentar fàcilment les capacitats de processament i emmagatzematge de dades.
- **Tolerància a fallades**: garanteix la disponibilitat del sistema, encara que es produeixin fallades en algunes de les màquines, evitant la pèrdua de dades.
- **Dades distribuïdes**: les dades han d'estar emmagatzemades entre diferents màquines evitant així el problema d'emmagatzemar grans volums de dades en un únic node central (*SPOF*).
- **Processament distribuït**: el tractament de les dades es realitza entre diferents màquines per millorar els temps d'execució i dotar el sistema d'escalabilitat.
- **Localització de la dada**: les dades a treballar i els processos que els tracten han d'estar a prop, per evitar les transmissions per xarxa que afegeixen latències i augmenten els temps d'execució.

 Generalitat de Catalunya Departament d'Educació i FP	CE IA&BD	
M5075 BigData Aplicat		Arquitectures de BigData

WAF

Els proveïdors *cloud* han definit una sèrie de principis i pràctiques a seguir a l'hora de dissenyar una arquitectura. Et recomano consultar la proposta d'AWS coneguda com a AWF ([AWS Well-Architected Framework](#)) o la d'Azure ([Azure Well-Architected Framework](#)).



Pilars de AWS Well-Architected Framework.

Abans de conèixer les arquitectures més utilitzades, és convenient tenir present sempre quin és l'objectiu que ha de complir la nostra solució. **És molt fàcil caure en la sobreenginyeria** i muntar una arquitectura amb una amalgama de productes que després són difícils de configurar i mantenir.

Per a això, és molt útil tenir al cap aquests principis:

1. **Escollir els components comuns amb suma cura**, com poden ser l'emmagatzematge dels objectes, el control de versions, les eines de monitoratge i orquestració, o els motors de processament.
2. **Planificar per a la decisió**. El CTO d'AWS, *Werner Vogels*, té una [cita](#) molt apropiada "*Everything fails, all the time.*" (tot falla, tota l'estona). Per això, hem de tenir en compte la disponibilitat i confiabilitat dels sistemes, així com els [objectius de temps \(RTO\)](#) i [objectiu de punts \(RPO\) de recuperació](#).
3. **Definir l'arquitectura per a l'escalabilitat**, tant feia amunt com feia avall, és a dir, que sigui elàstica (fins i tot en alguns casos, com poden ser els sistemes *serverless*, arribar fins a zero).

 Generalitat de Catalunya Departament d'Educació i FP	CE IA&BD	
M5075 BigData Aplicat		Arquitectures de BigData

4. **L'arquitectura és viva**, per tant, sempre ha d'evolucionar de la mà dels canvis tecnològics i de negoci.
5. **Buscar el baix acoblament entre sistemes**, per exemple, amb pas de missatges o ús de microserveis mitjançant APIs REST, de manera que, atesa una necessitat, puguem substituir un component per un altre. El baix acoblament permet modificar un component sense necessitat de pensar en les seves dependències.
6. **Facilitar tornar a un punt anterior** desfent una decisió errònia (i que sigui reversible), seguint el principi de les dues portes de *Jeff Bezos*.
7. **Prioritzar la seguretat**, mitjançant el principi de mínim privilegi (concedir únicament els privilegis d'usuari mínims que necessita l'usuari.) i el model de responsabilitat compartida.

Tipus d'arquitectures

Atès que les empreses disposen d'un volum de dades cada vegada més gran i la necessitat d'analitzar-les i obtenir-ne valor al més aviat possible, sorgeix la necessitat de definir noves arquitectures per cobrir casos d'ús diferents dels que hi havia fins al moment.

En aquesta sessió ens estem centrant en les arquitectures *Big Data* i no hem volgut barrejar els conceptes amb les arquitectures de dades com puguin ser un *Data Warehouse* o un *Data Lake*.

Des d'un punt de vista de l'arribada de les dades i el seu processament, les arquitectures més comunes en aquests projectes són principalment dues: ***Lambda*** i ***Kappa***. La principal diferència entre ambdues són els fluxos de tractament de dades que hi intervenen.

Un parell de conceptes que hem de definir abans de veure les característiques d'ambdues són el processament *batch* i el processament en *streaming*.

Definició de Processament

És la transformació de les dades perquè siguin viables per a l'anàlisi. Les dades poden venir de molts orígens diferents. Caldrà tractar-les i preparar-les.

Les dades en brut poden ser inconsistents, imprecises i repetitives. Les dades gairebé sempre es transformaran a mesura que avancen pel procés:

- Netejar o normalitzar les dades.
- Modificar el format per admetre una eina d'anàlisi específica.
- Augmentar el conjunt de dades omplint els buits o enriquint-lo amb informació addicional.
- Afegir metadades per categoritzar i catalogar les dades.

 Generalitat de Catalunya Departament d'Educació i FP	CE IA&BD	
M5075 BigData Aplicat		Arquitectures de BigData

Processament Batch

Batch fa referència a un procés en què **intervenen un conjunt de dades i que té un inici i un fi en el temps**. També se'l coneix com a processament per lots i s'executa sense control directe de l'usuari.

Per exemple, si tenim un conjunt de dades molt gran amb múltiples relacions, pot portar-nos de l'ordre d'hores executar les consultes que necessita el client, i per tant no es poden executar en temps real i necessiten algorismes paral·lels (com per exemple, *Map Reduce*). En aquests casos, els resultats s'emmagatzemen en un lloc diferent del d'origen per a posteriors consultes.

Un altre exemple, si tenim una aplicació que mostra el total de casos COVID que hi ha a cada ciutat, en comptes de realitzar el càlcul sobre el conjunt complet de les dades, podem realitzar una sèrie d'operacions que facin aquests càlculs i els emmagatzemen en taules temporals (per exemple, mitjançant `INSERT ... SELECT`), de manera que, si volem tornar a realitzar la consulta sobre totes les dades, accediríem a les dades ja calculades de la taula temporal. El problema és que aquest càlcul necessita actualitzar-se, per exemple, de manera diària, i d'aquí que calgui refer totes les taules temporals.

És el processament que s'ha realitzat des dels inicis del treball amb dades, tant a nivell de bases de dades com amb *Data Warehouses*. **Útil quan lo prioritari és afegir o transformar dades més que no pas fer-ne un anàlisi en temps real.**

De la mà del processament *batch* s'ha implantat l'ecosistema *Hadoop* amb totes les eines que abasten un procés ETL (extracció, transformació i càrrega de les dades). Aquests conceptes els treballarem més endavant.

Per exemple, passar de .csv a .parquet, ens permet accelerar l'anàlisi i estalvia emmagatzemament, cost i temps:

.CSV	.parquet
És el format més comú per emmagatzemar dades tabulars	Emmagatzema les dades en forma de columna
No és eficient per emmagatzemar o manipular grans quantitats de dades (més de 15 GB)	Està optimitzat per a l'emmagatzematge
	És adequat per al processament en paral·lel

 Generalitat de Catalunya Departament d'Educació i FP	CE IA&BD	
M5075 BigData Aplicat		Arquitectures de BigData

Processament en Streaming

Un processament és de tipus **streaming** quan està contínuament rebent i tractant un volum gran de nova informació segons va arribant sense tenir un fi pel que fa a l'apartat temporal.

Aquest processament es relaciona amb l'anàlisi en temps real. Per a això, s'utilitzen diferents sistemes basats en l'ús de cues de missatges.

No tot és temps real

No confondre temps real amb immediatesa. En informàtica, un sistema de temps real és aquell que respon en un període de temps finit, normalment molt petit, però no té per què ser instantani. Busquem la baixa latència, a diferència del processament per lots (batch)

Arquitectura Lambda

Representada mitjançant la lletra grega, va aparèixer l'any 2012 i s'atribueix a [Nathan Marz](#).

La va definir en base a la seva experiència en sistemes de tractament de dades distribuïdes durant la seva etapa com a empleat a les empreses *BackType* i *Twitter*, i està inspirada en el seu article [Com vèncer el teorema CAP](#).

El seu objectiu era muntar un sistema robust i tolerant a fallades, tant humans com de maquinari, que fos linealment escalable i que permetés realitzar escriptures i lectures amb baixa latència.

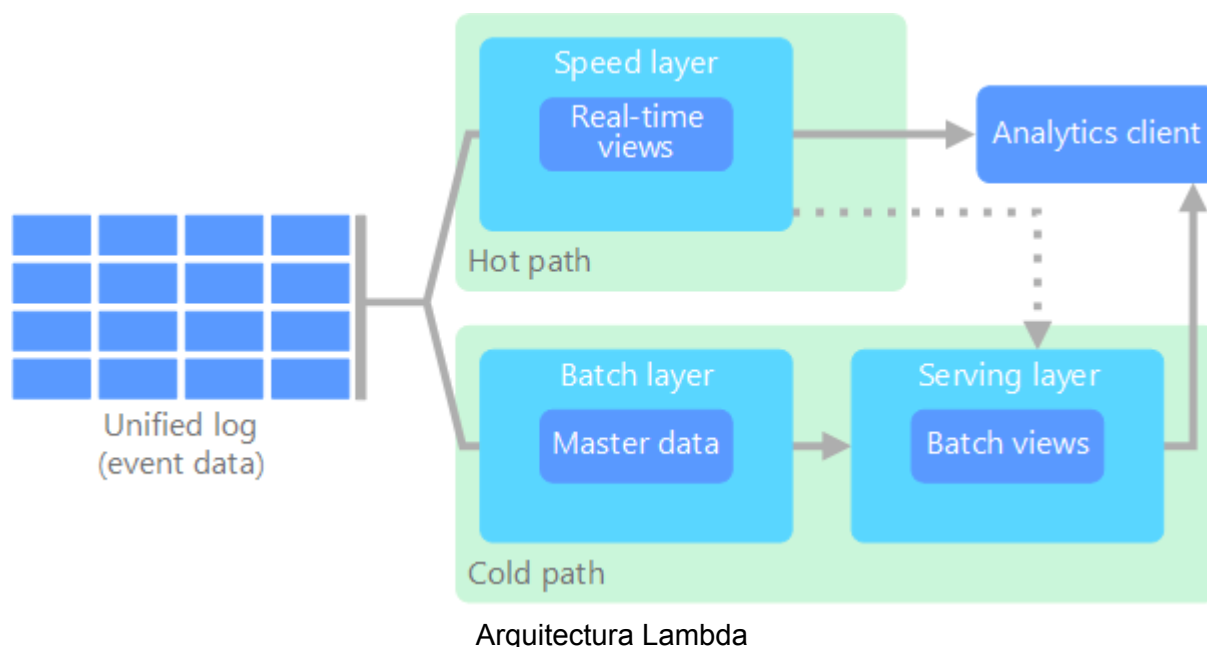
Totes les dades que arriben al sistema aniran per dos camins, un camí el lent (**capa batch**) i un altre camí el ràpid (**capa streaming**), que finalment conflueixen en la capa de consultes. Així doncs, es compon de tres capes:

- **Capa batch**: s'encarrega de gestionar les dades històriques i recalculer els resultats. De manera específica, la capa *batch* rep totes les dades en cru, **les emmagatzema de forma immutable** i les combina amb l'històric existent (s'afegeixen a les dades existents i les dades prèvies mai se sobreescrueixen) i recalcula els resultats iterant sobretot el conjunt de dades combinat. Qualsevol canvi en una dada s'emmagatzema com un nou registre, no modifica res, per així poder seguir el **llinatge de les dades**. (el llinatge/traçabilitat indica les transformacions que ha sofert una dada, des de l'origen, fins a l'estat actual, incloent-hi les combinacions amb altres dades o el canvi de la dada en si durant el seu cicle de vida)) Així doncs, aquesta capa opera sobre el conjunt complet i permet que el sistema produeixi els resultats més precisos. Tanmateix, això comporta un cost d'alta latència a causa dels requisits de temps de computació.

 Generalitat de Catalunya Departament d'Educació i FP	CE IA&BD	 institut sa palomera
M5075 BigData Aplicat		Arquitectures de BigData

- Capa de **streaming** / *speed*: serveix per oferir resultats amb **molt baixa latència**, proper al temps real. Aquesta capa rep les dades i realitza **modificacions incrementals** sobre els resultats de la capa *batch*. Gràcies als algoritmes incrementals implementats en aquesta capa, s'aconsegueix reduir el cost computacional de manera considerable, a cost de perdre alguna cosa de precisió.
- Capa de **serving**: permet la consulta dels resultats enviats des de les dues capes anteriors, en base a les vistes *batch* que omplen les capes anteriors.

Podem veure un esquema de l'arquitectura en el gràfic següent:



Temperatura de les dades

Depenent de la freqüència amb què s'accedeix a les dades, podem dividir el seu emmagatzematge en tres categories de persistència

	Calent	Tebi	Fred
Freqüència d' accés	molt freqüent	poc freqüent	molt poc freqüent
Cost d'emmagatzematge	alt	mitjà	baix
Cost de recuperació	baix	mitjà	alt
<i>Maquinari</i>	SSD, RAM	S3 poc freqüent	HDD, cinta, Glacera
Cas d'ús	Paginació de resultats, caixet de consultes	Generació de nòmnes	<i>backups</i>

 Generalitat de Catalunya Departament d'Educació i FP	CE IA&BD	
M5075 BigData Aplicat		Arquitectures de BigData

Les dades que flueixen per la capa de velocitat/*streaming* tenen la restricció de latència que imposa la mateixa capa per poder processar les dades tot el ràpid que sigui possible. Normalment, aquest requisit xoca amb la precisió de les dades.

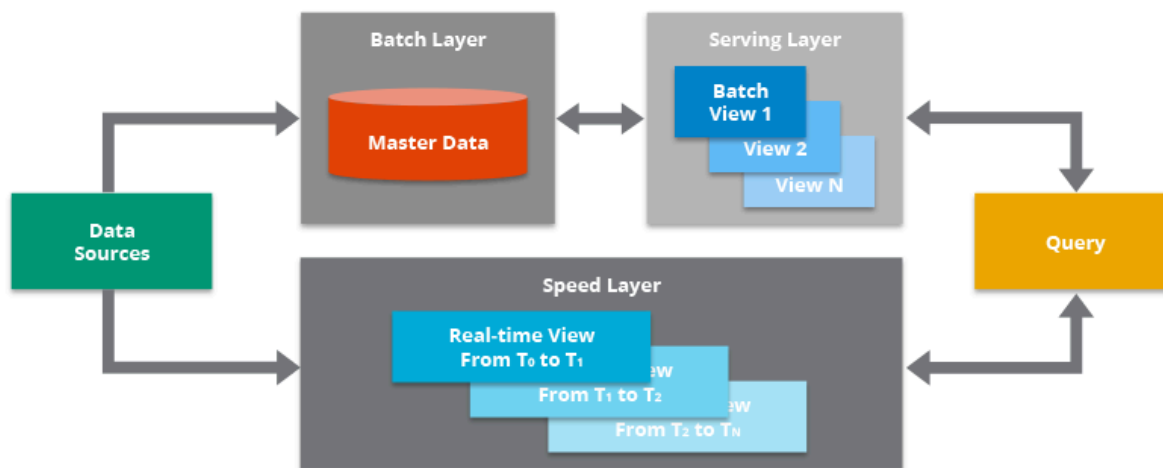
Per exemple, en un escenari IoT on es llegeixen un gran nombre de sensors de temperatura que envien dades de telemetria, la capa de velocitat es pot utilitzar per processar una finestra temporal de les dades que entren (per exemple, els deu primers segons de cada minut).

Les dades que flueixen pel camí lent no estan subjectes als mateixos requisits de latència, la qual cosa permet una major precisió computacional sobre grans conjunts de dades, que poden comportar molt de temps de processament.

Finalment, ambdós camins, el lent i el ràpid, convergeixen en les aplicacions analítiques del client. Si el client necessita informació constant (propera al temps real) tot i que menys precisa, obtindrà les dades del camí ràpid. Si no, ho farà a partir de les dades de la capa *batch*.

Altrament dit, el camí ràpid té les dades d'una petita finestra temporal, la qual es pot actualitzar amb dades més precises provinents de la capa *batch*.

Pas a pas



Arquitectura Lambda

El flux de treball és el següent:

1. La nova informació recollida pel sistema s'envia tant a la capa *batch* com a la capa de *streaming* (*Speed Layer* en la imatge anterior).
2. A la capa *batch* (*Batch Layer*) es gestiona la informació en cru, és a dir, sense modificar. Les dades noves s'afegeixen als ja existents. Seguidament, es fa un

 Generalitat de Catalunya Departament d'Educació i FP	CE IA&BD	
M5075 BigData Aplicat		Arquitectures de BigData

tractament mitjançant un procés *batch* el resultat del qual seran les *Batch Views*, que es faran servir a la capa que serveix les dades per oferir la informació ja transformada a l'exterior.

3. La capa que serveix les dades (*Serving Layer*) indexa les *Batch Views* generades en el pas anterior de manera que puguin ser consultades amb temps de resposta molt baixos.
4. La capa de *streaming* compensa l'alta latència de les escriptures que passa a la *serving layer* i només té en compte les dades noves (increments entre els processos batch i el moment actual).
5. Finalment, combinant els resultats de les *Batch Views* i de les vistes en temps real (*Real-time Views*), es construeix la resposta a les consultes realitzades.

Arquitectura Kappa

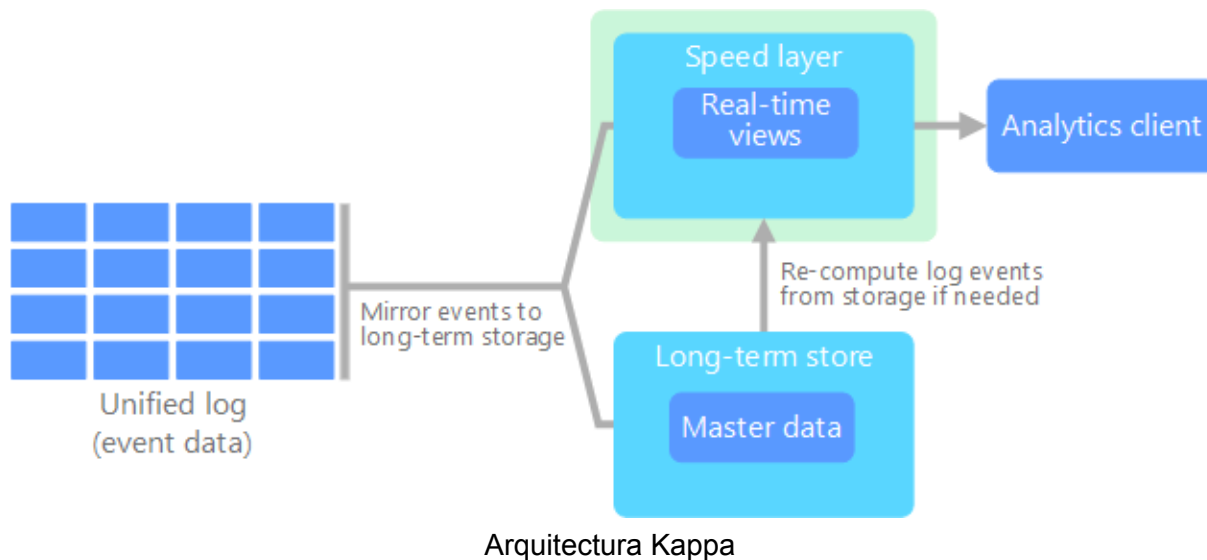
El terme va ser introduït el 2014 per [Jay Kreps](#) en el seu article [Qüestionant l'arquitectura lambda](#). Hi assenyala els possibles punts febles de l'arquitectura *Lambda* i com solucionar-los mitjançant una evolució.

Un dels majors inconvenients de l'arquitectura *Lambda* és la seva complexitat. El processament de les dades es realitza **en dos camins diferenciats**, la qual cosa comporta duplicar la lògica de computació i la gestió de l'arquitectura d'ambdós camins.

El que assenyala *Jay Kreps* en la seva proposta és que totes les dades flueixin per un únic camí, **eliminant la capa *batch* i deixant només la capa de *streaming*.** Aquesta capa, a diferència de la de tipus *batch*, no té un començament ni un fi des d'un punt de vista temporal i està contínuament processant noves dades a mesura que van arribant.

Per aconseguir un únic camí, se substitueixen les fonts de dades per cues de missatges (per exemple, mitjançant una eina com [Apache Kafka](#)), obtenint fluxos de dades que faciliten l'accés en temps real, llegint i transformant les dades dels missatges en un format que sigui fàcilment accessible als usuaris finals.

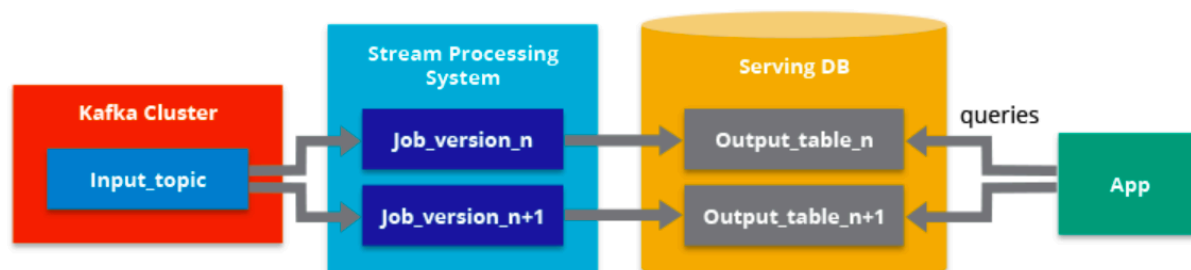
 Generalitat de Catalunya Departament d'Educació i FP	CE IA&BD	
M5075 BigData Aplicat		Arquitectures de BigData



Podem dir que els seus quatre pilars principals són els següents:

1. Tot és un ***stream***: les operacions *batch* són un subconjunt de les operacions de *streaming*, per la qual cosa tot pot ser tractat com un *stream*.
2. Les dades de partida no es modifiquen: les dades s'emmagatzemen sense ser transformades, per tant, són immutables, i les vistes se'n deriven. Un estat concret pot ser recalculat atès que la informació d'origen no es modifica.
3. Només hi ha un flux de processament: ja que mantenim un sol flux, el codi, el manteniment i l'actualització del sistema es veuen reduïts considerablement.
4. Tenim la possibilitat de tornar a llançar un processament: es pot modificar un processament concret i la seva configuració per variar els resultats obtinguts partint de les mateixes dades d'entrada.

 Generalitat de Catalunya Departament d'Educació i FP	CE IA&BD	
M5075 BigData Aplicat		Arquitectures de BigData



Arquitectura Kappa mitjançant Kafka

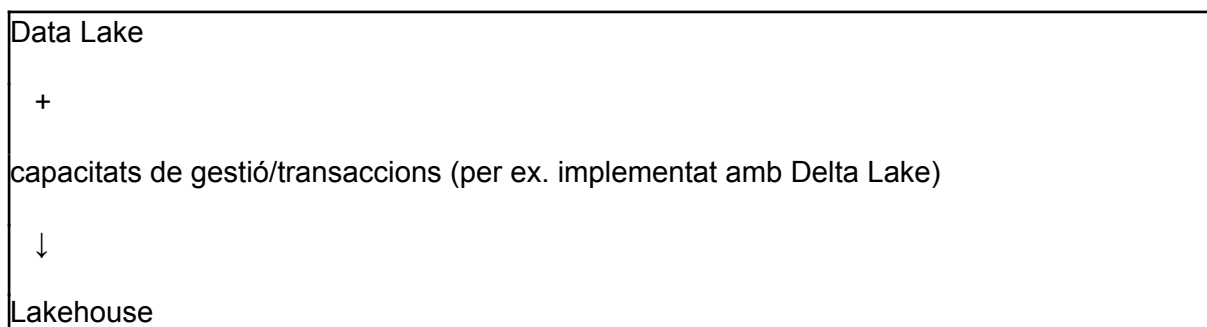
Com a requisit previ a complir, s'ha de garantir que els esdeveniments es llegeixen i emmagatzemen en l'ordre en què s'han generat. D'aquesta manera, podrem variar un processament concret partint d'una mateixa versió de les dades.

 Generalitat de Catalunya Departament d'Educació i FP	CE IA&BD	
M5075 BigData Aplicat		Arquitectures de BigData

Arquitectura Delta

Actualment, l'**arquitectura Delta** sembla ser el següent pas al voltant de les dades. Però primer, és millor estar familiaritzat amb el concepte de **Delta Lake**, tenint en compte que Delta es basa en això. **Delta Lake**, és una tecnologia d'emmagatzematge de codi obert que aporta compatibilitat amb transaccions ACID i aplicació d'esquemes a llacs de dades (Data Lakes) impulsats per **Apache Spark**.

Mitjançant Delta Lake, els usuaris poden construir un tipus d'arquitectura anomenat "Lakehouse" que treballa amb dades estructurades, semiestructurades i no estructurades sobre un Data Lake. Un Lakehouse és diferent d'un Data Lake i d'un Data Warehouse.



Delta Lake [amplia](#) els arxius de dades de Parquet amb un registre de transaccions basat en arxius per a transaccions ACID (Atomicity, Consistency, Isolation, Durability) i gestió de metadades escalable. També és compatible amb les API d'Apache Spark i està integrat amb Structured Streaming. A més, la separació entre les capes de l'arquitectura Delta és mínima en comparació amb l'arquitectura Lambda, per la qual cosa no és necessari tractar les dades de forma diferent en funció del seu origen.

Tenint en compte aquest context, **Databricks el presenta com "un enfocament completament diferent per a la ingesta, el processament, emmagatzematge i l'administració de dades centrat en la simplicitat**. Tot el processament i enriquiment de les dades, des de **Bronze** (dades sense processar) fins a **Silver** (filtrats) i **Gold** (totalment llestos per ser utilitzats per l'anàlisi, la generació d'informes i la ciència de dades) es produeix dins de Delta Lake, cosa que requereix menys salts de dades".

Les promeses de Delta Architecture

- **Redueixi els seus costos:** la seva simplicitat l'ajuda a reduir els costos significativament en reduir la quantitat de dades que s'han d'enviar i rebre, el temps necessari per processar les dades, així com la quantitat de vegades que necessita executar treballs a causa d'errors.

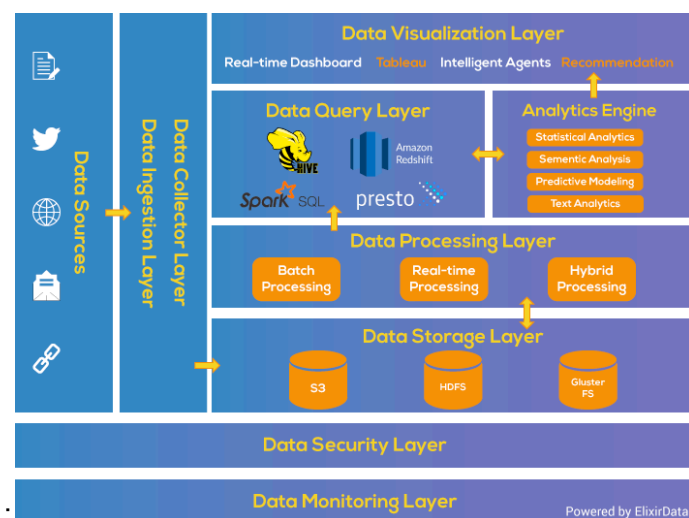
 Generalitat de Catalunya Departament d'Educació i FP	CE IA&BD	
M5075 BigData Aplicat		Arquitectures de BigData

- **Delta = Menys codi:** com ja vam dir, les arquitectures Lambda necessiten diferents bases de codi per a cada part de l'arquitectura. Però amb Delta, com que les transaccions són compatibles amb ACID, s'assegura que el seu codi sigui menys complex perquè diverses parts del codi que s'havien de fer manualment (per garantir la coherència de les dades, per exemple) ja no són necessàries.
- **Indexació millorada:** quan s'utilitza Delta Lake com a emmagatzematge per a l'arquitectura, es reuneixen les capacitats d'ús dels [índexs de filtre de Bloom](#), que milloren el rendiment de l'execució de consultes en més d'un 50%, segons [MSSQLTips.com](#).
- **Una font de dades:** quan s'utilitzen altres arquitectures i s'intenta simplificar els processos, les dades sovint es copien d'un llac de dades a altres magatzems de dades més petites. Això crea problemes de coherència i control de versions que es resolen mitjançant l'ús de l'arquitectura Delta.
- **Afegiràs més fonts de dades? No hi ha problema:** en general, després de dissenyar i implementar una arquitectura de dades per a un cas d'ús específic, és difícil que s'agreguin noves fonts de dades. Però quan es fa servir Delta Lake com a motor, això ja no presenta un desafiament enorme, ja que l'evolució [de l'esquema](#) fa que agregar noves fonts de dades (o canviar els formats de les fonts de dades existents) sigui una tasca més senzilla.

Arquitectura per capes

A més de les solucions que acabem de conèixer, una altra forma de dissenyar les capes d'una arquitectura *Big Data* consisteix a separar les diferents fases del [cicle de vida de la dada](#) en capa diferenciades.

L'arquitectura [per capes](#) dona suport tant al processament *batch* com per *streaming*.



 Generalitat de Catalunya Departament d'Educació i FP	CE IA&BD	
M5075 BigData Aplicat		Arquitectures de BigData

Arquitectura per capes (xenonstack.com)

L'arquitectura consisteix en 6 capes que asseguruen un flux segur de les dades

- Capa **d'ingesta**: és la primera capa que recull les dades que provenen de fonts diverses. Les dades es categoritzen i prioritzen, facilitant el flux d'aquests en posteriors capes.
- Capa de **col·lecció**: Centrada en el transport de les dades des de la ingesta a la resta del *pipeline* de dades. En aquesta capa les dades es desfan per facilitar l'analítica posterior.
- Capa de **processament**: Aquesta és la capa principal. Es processen les dades recollides en les capes anteriors (sigui mitjançant processos batch, streaming o models híbrids), i es classifiquen per a decidir feia quina capa es dirigeix.
- Capa **d'emmagatzematge**: Se centra a decidir on emmagatzemar de forma eficient l'enorme quantitat de dades. Normalment en un magatzem d'arxius distribuït, que dona peu al concepte de *data lake*.
- Capa de **consulta**: capa on es realitza el processament analític, centrant-se a obtenir valor a partir de les dades.
- Capa de **visualització**: també coneguda com a capa de presentació, és amb la qual interactuen els usuaris.

Tecnologies

Per exemple, la **ingesta de dades** cap a les arquitectures *Lambda* i *Kappa* es poden realitzar mitjançant un sistema de missatgeria de **cues publicador/subscriptor** com **Apache Kafka** i/o un **servei de flux de dades** com **Apache Nifi**.

L'**emmagatzematge de les dades** i models el podem realitzar mitjançant **HDFS** o **S3**. Dins d'una arquitectura *Lambda*, en el sistema batch, mitjançant algorismes *MapReduce* de *Hadoop* o Consultes *Hive* podem generar models. Per a la capa Streaming (tant per a *Lambda* com *Kappa*) es poden utilitzar altres tecnologies com [Apache Storm](#), [Apache Samza](#) o *Spark Streaming* per modificar models de forma incremental.

De forma alternativa, Apache Spark es pot utilitzar com a plataforma comuna per desenvolupar les capes *batch* i *streaming* de l'arquitectura *Lambda*. D'aquí la seva àmplia acceptació i ús el dia d'avui a la indústria, es codifica una vegada i es comparteix en ambdues capes.

La capa de *serving* es pot implementar mitjançant una base de dades NoSQL com pugui ser [Apache HBase](#), MongoDB, [Redis](#) o Base de dades d'AWS Dynamo. També es poden utilitzar motors de consultes com [Apache Drill](#) o [Presto](#).

 Generalitat de Catalunya Departament d'Educació i FP	CE IA&BD	
M5075 BigData Aplicat		Arquitectures de BigData

Informe Matt Turck

Cada any, *Matt Turck* publica un informe amb l'amalgama d'eines i tecnologies agrupades per camp existents a l'escenari de ML, AI i Dades que recomanem consultar: [The 2024 MAD \(Machine Learning, AI and Data\) Landscape](#) o també la del 2025 que pots consultar en aquest [link](#).

Principi SCV

En la sessió de NoSQL vam estudiar el Teorema de CAP (consistència, disponibilitat, tolerància a la partició), el qual se centra en l'emmagatzematge de dades distribuïdes. Si ens centrem en el **processament distribuït de les dades**, tenim el [principi SCV](#).

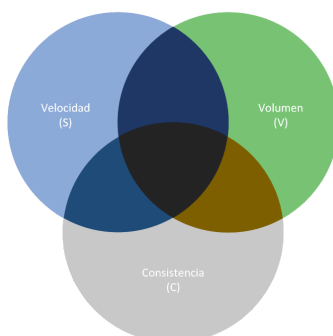
Aquest principi no té a veure amb l'escriptura i lectura (consistent o no) de les dades en entorns distribuïts sinó amb el processament que es realitza sobre ells dins dels nodes d'un sistema de processament distribuït.

De manera similar al que ocorria amb el *Teorema de CAP*, el principi SCV estableix que un sistema de processament distribuït només pot suportar com a màxim 2 de les següents 3 característiques:

- **Velocitat (*Speed*)**: Es refereix a quan tarden a processar-se les dades des del moment en què són rebudes en el sistema analític. En general, s'exclou el temps que es triga a capturar les dades, considerant només el que es triga a generar l'estadística o executar l'algorisme en qüestió. Aquesta velocitat és més alta si estem davant d'un sistema d'analítica en temps real que si es tracta d'un sistema d'analítica per lots (de l'anglès batch).
- **Consistència (*Consistency*)**: Es refereix en aquest cas a la **precisió dels resultats** de l'analítica (no confondre, per tant, amb el significat de la C del teorema CAP). Aquesta precisió depèn de si per a l'analítica s'utilitzen totes les dades disponibles (precisió alta) o de si, per contra, s'empren tècniques de mostreig per seleccionar només un subconjunt dels mateixos amb la intenció de produir resultats (de menor precisió) en un menor temps.
- **Volum (*Volum*)**. Es refereix a la quantitat de dades que poden ser processats. Cal tenir en compte que en entorns de *Big Data*, l'alt volum de dades és una característica sempre present (una de les 5 Vs).

De la mateixa manera que fem en estudiar el [teorema CAP](#), ens fixarem en una sèrie d'escenaris per mostrar que no podem aconseguir un sistema que compleixi alhora les 3 característiques del principi SCV.

 Generalitat de Catalunya Departament d'Educació i FP	CE IA&BD	
M5075 BigData Aplicat		Arquitectures de BigData



Principi SCV

- Si es requereix velocitat (S) i consistència (C), no podem processar un alt volum (V) de dades, ja que això augmenta el temps de resposta.
- Si es requereix consistència (C) i poder processar grans volums de dades (V), no és possible realitzar tal processat a una alta velocitat (S).
- Si necessitem processar un alt volum de dades (V) a una alta velocitat (S), llavors necessitarem emprar tècniques de mostreig per seleccionar només un subconjunt d'aquestes dades, la qual cosa produirà un resultat no consistent (C).

Casos d'ús

Quina arquitectura s'adapta millor als requisits que ens trasllada el client? ¿Lambda o Kappa? Quin encaixa millor en el nostre model de negoci?

Depèn. L'arquitectura *Lambda* és més versàtil i és capaç de cobrir un major nombre de casos, molts d'ells que requereixen fins i tot processament en temps real.

Una pregunta que ens hem de plantejar és, l'anàlisi i el processament (els seus algorismes) que vam realitzar a les capes *batch* i *streaming* és el mateix? En aquest cas l'opció més encertada seria l'arquitectura *Kappa*.

No obstant això, en altres ocasions necessitarem accedir a tot el conjunt de dades sense penalitzar el rendiment, per la qual cosa la *Lambda* pot ser més apropiada i fins i tot més fàcil d'implementar.

També ens inclinarem cap a *Lambda* si els nostres algorismes de *batch* i *streaming* generen resultats molt diferents, com pot succeir amb operacions de processament pensant o en models de *Machine Learning*. En aquests casos, els algorismes *batch* es poden optimitzar, ja que accedeixen al dataset històric complet.

Decidir entre *Lambda* i *Kappa* al final és una decisió entre afavorir el rendiment d'execució d'un procés *batch* sobre la simplicitat de compartir codi per a ambdues capes.

 Generalitat de Catalunya Departament d'Educació i FP	CE IA&BD	
M5075 BigData Aplicat		Arquitectures de BigData

Casos reals

Un cas d'ús real per a una **arquitectura *Lambda*** podria ser un sistema que recomani pel·lícules en funció dels gustos dels usuaris. D'una banda, tindria una capa batch encarregada d'entrenar el model i anar millorant les prediccions; i de l'altra, una capa streaming capaç d'encarregar-se de les valoracions en temps real.

Un exemple real d'una **arquitectura *Kappa*** seria un sistema de geolocalització d'usuaris per la rodalia a una antena de telefonia mòbil. Cada vegada que s'aproximi a una antena que li donés cobertura es generaria un esdeveniment. Aquest esdeveniment es processaria a la capa de *streaming* i serviria per pintar sobre un mapa el seu desplaçament respecte a la seva posició anterior.

Com a lectura recomanable tenim un parell de casos desenvolupats per *Ericsson* que podeu llegir en el següent [link](#).

És molt important sempre tenir al cap el ràpid que evolucionen els casos d'ús que volem cobrir i el mercat del *Big Data*, la qual cosa implica la necessitat d'adaptar-s'hi al més aviat possible, modificant l'arquitectura sobre la marxa.

Bones pràctiques

- **En la ingesta de dades:** avaluar els tipus de fonts de dades, no totes les eines serveixen per a qualsevol font de dades, i en algun cas el millor és combinar diverses eines per cobrir tot el ventall.
- **En el processament:** analitzar si el sistema ha de ser *streaming* o *batch*. Alguns sistemes que no es defineixen com a purament *streaming*, és a dir, utilitzen el que anomenen *micro-batch* que sol donar resposta a problemes que en l'ús quotidià del llenguatge es denomina com a *streaming*.
- **En el monitoratge:** en treballar amb multitud de tecnologies durant la ingesta és important utilitzar una eina per controlar, monitoritzar i gestionar l'arquitectura.
- Algunes decisions que hem de prendre a l'hora de triar l'arquitectura són:
 - **Enfocar els casos d'ús.** Quan tinguem els objectius clars sabrem quina part hem d'enfortir en l'arquitectura. ¿Volum, varietat, velocitat?
 - **Definir l'arquitectura:** ¿*batch* o *streaming*? Realment cal que la nostra arquitectura suporti *streaming*?
 - **Avaluar les fonts de dades:** Com d'heterogènies són? ¿Suporten les eines triades tots els tipus de fonts de dades que s'utilitzen?

 Generalitat de Catalunya Departament d'Educació i FP	CE IA&BD	
M5075 BigData Aplicat		Arquitectures de BigData

- **Arquitectura moderna:** El principal objectiu d'una arquitectura de *Big Data* moderna s'ha de centrar a unificar les diferents fonts de dades en un únic lloc que sigui la font veraç de les nostres dades.

 Generalitat de Catalunya Departament d'Educació i FP	CE IA&BD	
M5075 BigData Aplicat		Arquitectures de BigData

Referències

- [Big Data Lambda Architecture](#) per *Nathan Marz*.
- [Què és l'arquitectura lambda?](#)
- [Big Data Architectures](#) en la documentació d'Azure
- Llibre [Fonaments d'Enginyeria de Dades](#)
- [Arquitectura Lambda vs Arquitectura Kappa](#)
- [Arquitectures de processament de dades – Lambda i Kappa](#)
- [Arquitectura de Big Data: explicació detallada](#)